Building a Graph Analytics Pipeline: From Docker Containers to Distributed Stream Processing

Nithya Shree Ravikumar
Arizona State University

Abstract

The goal of this project is to design and develop a graph data processing platform that gradually matures from an isolated containerized framework into a fully distributed streaming solution. By leveraging the NYC yellow taxi dataset as the practical application scenario, we will construct a two-stage pipeline covering some of the basic fundamentals of graph databases, containerization, and stream processing. In Phase 1, we will set up a Neo4j graph database in a Docker container and apply various graph algorithms, such as PageRank and BFS. In Phase 2, we will further expand our work to a more production-like environment with the help of Kubernetes, Apache Kafka, and Kafka Connect.

1 Introduction

Graph databases are gaining popularity due to their capability to handle large-scale connected datasets in various fields, including social networks, logistics, recommendation engines, and financial fraud prevention. Graph databases differ from traditional relational databases in that they do not require expensive join operations to analyze connections, but rather treat connections as first-class citizens.

The aim of this project is to design and implement a pipeline capable of processing both batch and stream data in a graph-oriented manner. Specifically, we will use the NYC yellow taxis dataset for demonstration purposes, where each row represents taxi trip records containing geographic location details of pick-up and drop-off locations. We create an abstraction whereby taxi zones are nodes and taxi trips become edges connecting nodes, thereby forming a transportation graph.

This project involves two main stages with separate goals. In stage 1, we seek to create a reproducible development environment using Docker technology. Here, we validate our proposed data schema and run graph algorithms, namely PageRank for node centralities and Breadth-First Search (BFS) for shortest paths.

The second phase circumvents the limitations of the monolithic approach in terms of scalability through the transition to a microservice-based distributed architecture. Our deployment is achieved using a local Kubernetes cluster provided by Minikube, where we integrate Apache Kafka as an event stream processing layer and Kafka Connect for integrating Neo4j with a declarative sink API.

Contributions of this study include: showcasing the evolution of the architecture from proof of concept to production implementation, performing graph analytics on realistic transportation data; providing patterns for the integration of a graph database with a stream processing platform, and applying container orchestration techniques with Kubernetes.

2 Methodology

2.1 System Architecture Overview

Figure 1 depicts the overall architecture of the pipeline throughout both phases. Phase 1 (upper part) depicts the Docker based workflow from the ingest phase, graph loading, and algorithm invocation. Phase 2 (lower part) depicts the Kubernetes based streaming capability extension, wherein Kafka is utilized as the event bus and Kafka Connect as the declarative sink to Neo4j.

2.2 Phase 1: Dockerized Graph Processing Environment

Phase 1 lays the groundwork for the infrastructure through the creation of containers for each component used for graph analytics.

2.2.1 Container Architecture

This container consists of an Ubuntu operating system version 22.04. This Docker container comprises OpenJDK 21 for the execution of Java, Python 3, and the following libraries: `neo4j`, which is used to connect to the Neo4j database; `pandas`, which is used for handling structured data; and `pyarrow` for Parquet files.

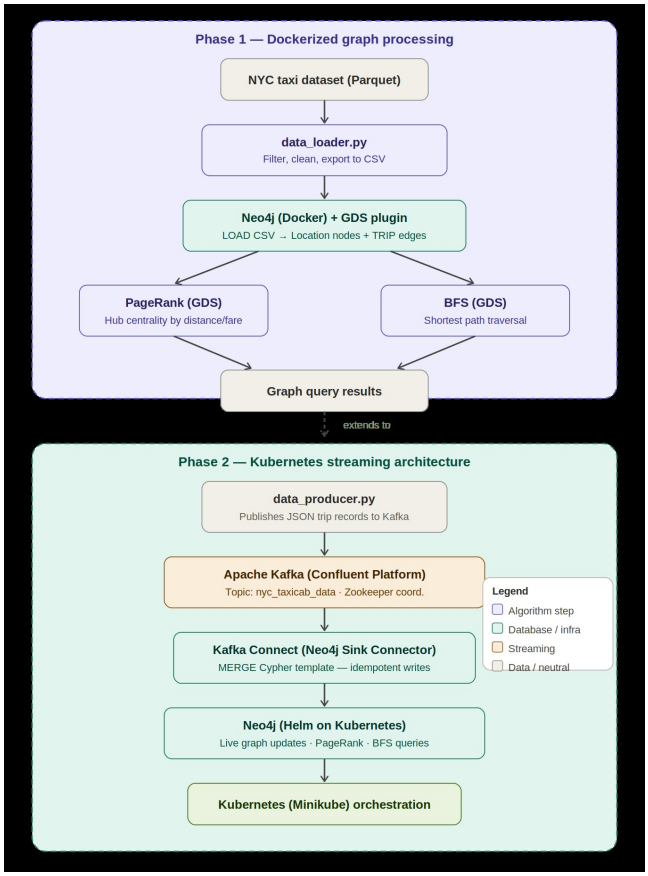


Figure 1: Full system architecture: Phase 1 (Dockerized graph processing) and Phase 2 (Kubernetes streaming pipeline). Arrows indicate data flow direction.

Neo4j is used as the graph database engine at the heart of our analytics workflow. The Graph Data Science (GDS) plugin version 2.21.0 is downloaded and installed, providing optimized and production-ready algorithms for the purpose. The GDS JAR file is downloaded from the Neo4j repository and is put in `/var/lib/neo4j/plugins/`.

The configuration is done automatically within the Dockerfile to ensure reproducibility. This involves editing the `/etc/neo4j/neo4j.conf` file to allow binding Neo4j to all IP addresses on the system and whitelisting APOC and GDS procedures. It also involves setting the default password to `graphprocessing` using Neo4j's built-in tools. The working directory is `/cse511/`.

2.2.2 Data Ingestion Pipeline

The input data is the NYC yellow taxi data from March 2022 in Parquet format, which is a column-based storage format that allows efficient I/O of large files.

A `data_loader.py` utility runs the entire ingestion process. It waits until Neo4j completes initialization (usually after 2–4 minutes from starting the Docker container), reads the file through PyArrow, preprocesses data by selecting necessary columns (timestamp of pickup and dropoff, zone id, trip distance, and fare amount) and applying geographic filters to keep only required boroughs. Anomalies, like very small trip distance or fare amounts, are excluded as possible errors.

Preprocessed data is exported in CSV format and imported using Neo4j's `LOAD CSV` operation. Ingestion Cypher query creates `Location` nodes by zone id and `TRIP` relations with the following properties: `distance` (`float`), `fare` (`float`), `pickup_dt` (`datetime`), and `dropoff_dt` (`datetime`).

2.2.3 Graph Algorithm Implementation

Two algorithms for graphs are provided to exhibit two different analytical approaches towards the transportation network.

PageRank Algorithm: The PageRank algorithm is a method of assessing the relevance of nodes by passing around score values within the network depending on their incoming edges. In the context of the transportation network, high scoring nodes represent those that are highly active in terms of pickups and drop-offs. The PageRank function available within the GDS library is called, where the distance or fare attribute is set as the edge weight. After several iterations, nodes with the highest and lowest scores are returned.

Breadth-First Search: BFS involves searching a graph by traversing nodes according to their hop distance from the starting node. The breadth-first search technique is implemented to find the shortest paths between locations. In order to accomplish this, the built-in shortest path functions of Neo4j are called.

2.3 Phase 2: Kubernetes-Based Streaming Architecture

In Phase 2, the container is broken down into a number of independent, cooperating services, which can then be run on Kubernetes, allowing real time ingestion.

2.3.1 Container Orchestration Setup

Minikube is used to create an identical Kubernetes environment to our production environment locally in one development machine. The components of each system are represented by Kubernetes pods based on specifications such as resource limits, ports, and other settings provided in YAML files.

2.3.2 Message Streaming Infrastructure

The Apache Kafka event bus is responsible for creating a durable and scalable message passing system between producers and consumers.

Apache Zookeeper is installed first because Kafka needs it for its coordination processes. The `zookeeper-setup.yaml`

configuration starts Zookeeper as a persistent deployment and makes it accessible at `zookeeper-service:2181` in the Kubernetes cluster.

The Kafka broker is deployed using the Confluent Platform image (`confluentinc/cp-kafka:7.3.3`). Key environment variables include `KAFKA_BROKER_ID`, `KAFKA_ZOOKEEPER_CONNECT`, `KAFKA_ADVERTISED_LISTENERS` (exposing both an external listener at `localhost:9092` and an internal listener at `kafka-service:29092`), and `KAFKA_AUTO_CREATE_TOPICS_ENABLE` to allow on-demand topic creation. The dual-listener configuration allows external Python scripts to publish messages while services inside the cluster communicate over the internal interface.

2.3.3 Graph Database Deployment

Neo4j is installed using Helm charts that package the entire installation process within one configuration. The `neo4j-values.yaml` file configures the Neo4j installation by enabling the GDS plug-in, setting the password to `processingpipeline`, and specifying CPU and memory limits. There is also a `neo4j-service.yaml` file that allows access to the Bolt interface on port 7687 and the HTTP interface on port 7474 with the hostname of `neo4j-service`.

2.3.4 Stream Processing Integration

With Kafka Connect, it becomes unnecessary to code consumers since the framework provides declarative sinks. A Docker image (`veedata/kafka-neo4j-connect`) containing the Neo4j Sink Connector is used. The image has an initialization script that waits for the availability of the REST API of Kafka Connect, configures Kafka topics with log compaction, and registers the Neo4j Sink connector using the REST API. The connector makes use of a Cypher `MERGE` pattern to consume data idempotently without altering the graph when messages are delivered twice.

2.3.5 Data Producer Implementation

A Python-based producer script (`data_producer.py`) generates real-time trip events by consuming the preprocessed data and publishing each trip event in JSON format on the `nyc_taxicab_data` Kafka topic. Intentional message delay helps in regulating the data flow rate and preventing any overload at the consumer end.

3 Results

3.1 Phase 1: Containerized System Validation

The Docker-based implementation proved itself reliable in demonstrating basic graph analytics functionalities. Neo4j was always up and running within 2 to 4 minutes after container initialization. Figure 2 presents the login page of Neo4j Browser that can be accessed through `localhost:7474`.

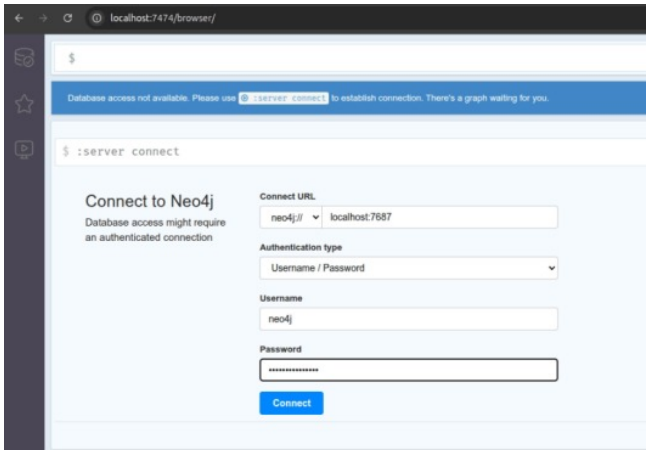


Figure 2: Neo4j Browser login screen at `localhost:7474/browser`, confirming successful container startup and database availability.

It took less than five minutes for the filtered database having around 50,000 trips to load successfully. The schema of the graph has been verified through the use of `CALL db.schema.visualization()` and is illustrated in Figure 3. The correct `Location` and `TRIP` types are

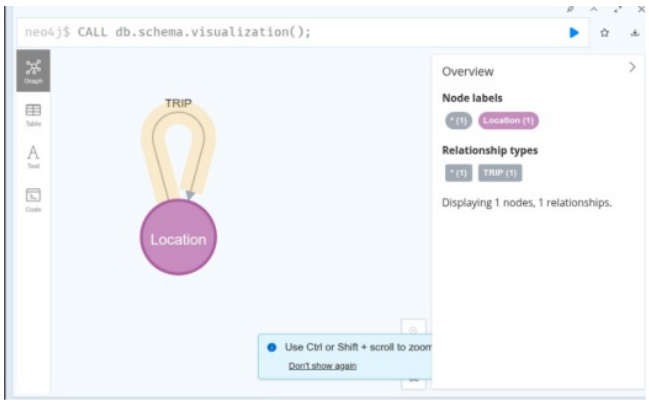


Figure 3: Neo4j schema visualization output (`CALL db.schema.visualization()`), showing one `Location` node type and one `TRIP` relationship type.

The performance of the queries was good throughout. Single hop searches took less than 100 milliseconds to complete, whereas multi-hop traversals took 200–500 milliseconds based on the number of hops.

PageRank Results: The process of convergence took around 2–3 seconds with the default configuration settings. Locations that were closer to important transportation hubs and business districts were assigned higher ranks by the

distance-weighted version of PageRank. Remote locations with few connections were assigned low PageRank values, which is expected within the given domain.

BFS Results: Paths were generated successfully between any two locations. The most common number of hops for a shortest path was 2–4.

Figure 4 shows the result of `MATCH (n) RETURN n LIMIT 25`, rendering 25 `Location` nodes and their dense `TRIP` relationship network in the Neo4j browser. The overview panel confirms 873 `TRIP` relationships in the loaded graph.

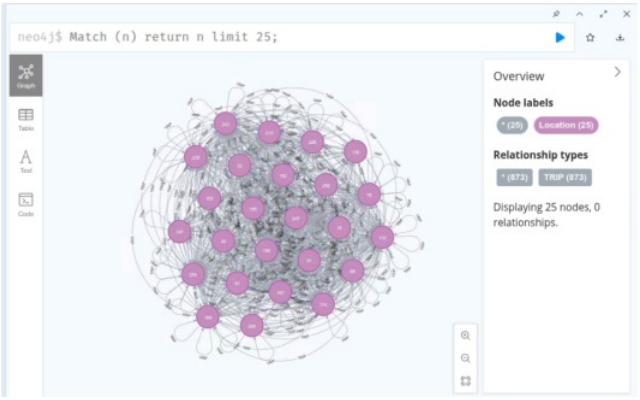


Figure 4: Neo4j Browser graph view from `MATCH (n) RETURN n LIMIT 25`, displaying 25 `Location` nodes with 873 `TRIP` relationships visualized as a dense network.

3.2 Phase 2: Distributed System Performance

All the components of Kubernetes entered into the `Running` state with no issues by using `kubectl get pods -A` command.

Message Streaming: The publisher sent messages at the rate of 10-20 messages/second. No message loss was observed during streaming, and consumer lag did not exceed 100 messages throughout testing.

Latency: The latency between message publication on Kafka and commit on Neo4j was between 1-2 seconds. Such latency is adequate for an analytical application.

Integrity Check: Several thousands of messages were successfully streamed into Neo4j and a check was carried out using Neo4j queries, verifying the number of nodes against unique location ID and the number of relationships with the number of published messages.

Correctness in Live Data: Both PageRank and BFS algorithms returned accurate results with an increasing graph size. Scores in PageRank changed smoothly upon receipt of new data, while some of the newly created `TRIP` links showed that there were faster BFS paths discovered. Latency stayed consistent during the test period.

Failure Detection and Recovery: Pod-level failures caused by manual shutdown led to instantaneous Kubernetes-based pod replacement, allowing services to recover quickly without losing any data.

4 Discussion

4.1 Architectural Evolution

The transition from Phase 1 to Phase 2 is characteristic of the typical development process in distributed systems. The use of containers helped accelerate development, but scaling was not possible, nor were there fault boundaries between components.

In Phase 2, however, all these issues are addressed using the technique of service decomposition. Kafka is responsible for ensuring event durability, Kafka Connect processes data transformations, while Neo4j takes care of graph storage and querying. Each component scales independently according to the detected chokepoints. Another advantage provided by Kafka is that the persistent message log allows replaying events, facilitating database restoration, testing different models of data organization, and disaster recovery.

4.2 Technology Selection Rationale

Neo4j: The use of the Cypher query language and the complete GDS plugin makes Neo4j an ideal tool for efficient graph analytics.

Kafka: While direct streaming to Neo4j offers simplicity, it fails to include replay functionality, support for multiple consumers, and decoupling of producers from databases. Kafka supports all three.

Kafka Connect: Using custom consumers demands writing lots of code to handle offset management, fault tolerance, and backpressure. Kafka Connect accomplishes that declaratively.

Kubernetes: Although Docker Compose works fine for local development, Kubernetes can help with automatic restarting and horizontal scaling which are needed in production.

4.3 Performance Considerations

PageRank is an iterative algorithm that scales non-linearly as the graph grows. We need to consider geographical partitioning and iterative update mechanisms for larger production graphs.

Kafka processing speed relies on the number of partitions and consumers concurrently consuming data. The current approach of using a single partition works fine for development purposes but won't scale in a production environment.

The more transactions grouped into one, the faster Neo4j writes become. Increasing the Kafka Connect batch size might result in better throughput with some latency trade off.

4.4 Future Enhancements

Full Network Analysis: Extending the approach to the entire network of taxis in NYC would test the architecture, which would involve distributed partitioning of graphs and federated querying.

Temporal Graph Analysis: Keeping time-stamped graph snapshots would facilitate analyzing changes in transportation patterns during different hours, days, or seasons.

Machine Learning Integration: The generated graph embeddings using the GDS library could be used as input features for demand forecasting or ride-hailing models.

Advanced Algorithm Implementation: Community detection would help reveal natural neighborhoods, whereas betweenness centrality would locate potential bottlenecks likely to become congested points.

Observability: The inclusion of Prometheus for collecting metrics and Grafana for dashboards would make it possible to have real-time visibility of the application in production.

Hardening Security: For production deployment, we need authentication, TLS for securing service-to-service communication, and Kubernetes networking policies for limiting pod-to-pod communications.

5 Conclusion

In summary, this project successfully implements a graph analytics platform, ranging from containerized proof-of-concept to full-scale streaming solution. Having gone through two well-formulated phases, we managed to solve the tasks of ensuring analytical correctness (Phase 1) and operational scalability (Phase 2).

Main results achieved in this project include creating a reproducible container-based environment, performing computations on a transportation graph, developing a scalable streaming-graph analytics architecture, and applying best practices in container orchestration using Kubernetes.

A live data stream of NYC taxi trips is fed into our graph analytics platform, which produces an up-to-date graph used for shortest path calculation and centrality measurement. Our solution offers flexibility by allowing separate scaling of different service layers, while the persistence of Kafka's log enables reliability and replayability.

Patterns of architecture design considered here, including graph modeling, stream-based input handling, and microservices composition, can be generalized to other industries such as social network analysis, fraud detection, recommendations, and logistics. Moving from Docker to Kubernetes, batch to stream, and monolithic to microservices architectures represents a natural evolutionary path for any production-ready data system.

Security improvement and performance tuning at production level constitute priorities for future development.

Acknowledgments

This project is part of coursework for CSE 511: Data Processing at Scale, offered by Arizona State University. The requirements and instructions for the project have been provided by the course instructors.

References

[1] Neo4j, Inc., "Neo4j Graph Data Science Library Documentation," https://neo4j.com/docs/graph-data-science/

[2] Neo4j, Inc., "Neo4j Operations Manual," https://neo4j.com/docs/operations-manual/

[3] Neo4j, Inc., "Neo4j Kubernetes Deployment Guide," https://neo4j.com/docs/operations-manual/current/kuber

[4] Apache Software Foundation, "Apache Kafka Documentation," https://kafka.apache.org/documentation/

[5] Neo4j, Inc., "Kafka Connect Neo4j Connector Documentation," https://neo4j.com/docs/kafka/

[6] Cloud Native Computing Foundation, "Kubernetes Documentation," https://kubernetes.io/docs/

[7] Kubernetes SIGs, "Minikube Documentation," https://minikube.sigs.k8s.io/docs/

[8] Cloud Native Computing Foundation, "Helm Documentation," https://helm.sh/docs/

[9] Apache Arrow, "PyArrow Documentation," https://arrow.apache.org/docs/python/

[10] pandas development team, "pandas Documentation," https://pandas.pydata.org/docs/

[11] Confluent, Inc., "Confluent Platform Documentation," https://docs.confluent.io/

[12] Docker, Inc., "Docker Documentation," https://docs.docker.com/

[13] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab, Technical Report, 1999.

[14] NYC Taxi and Limousine Commission, "TLC Trip Record Data,"